SYSTEM DESIGN IN 100 DAYS

The World’s Best System Design Course

Written in Simple Language - Like Teaching a 14-Year-Old

Author: AI Course Designer
Level: Beginner to Expert
Style: Simple, Fun, with Real Examples & Memory Tricks

TABLE OF CONTENTS

HOW TO USE THIS BOOK

1. Read ONE day per day - Don’t rush!
2. Draw diagrams - Use pen and paper
3. Explain to a friend - If you can teach it, you know it
4. Use the memory tricks - They help you remember forever
5. Do the mini-project at end of each week

PHASE 1: FOUNDATION (Days 1-20)

Building Your System Design Superpowers

DAY 1: What is System Design?

The Big Idea

System Design is like being an architect for software. Just like an architect designs a building (where the doors go, how many floors, where the elevator is), a system designer decides how software is built.

Real Life Example

Think about Instagram:
* Where do all the photos go? (Storage)
* How do millions of people see photos at the same time? (Servers)
* How does your feed load so fast? (Caching)
* What happens if one computer breaks? (Reliability)

THAT is system design!

Memory Trick

S.Y.S.T.E.M = Serve Your Software To Everyone Magnificently

Key Terms Today

Term | Simple Meaning
Server | A powerful computer that serves data to users
Client | Your phone/laptop that asks for data
Database | A giant organized notebook that stores information
Network | The road between your phone and the server

Draw This

```
[Your Phone] ---internet---> [Server] ----> [Database]
   (Client)                  (Brain)       (Memory)
```

Today's Quiz

1. What does a server do?

2. Name 3 things you'd need to design for Instagram.
3. What's the difference between a client and a server?

DAY 2: Client-Server Architecture

The Big Idea

Every app you use works like a restaurant:
* You (Client) = The customer
* Waiter (Internet) = Carries your order
* Kitchen (Server) = Makes your food
* Pantry (Database) = Stores ingredients

How It Works

1. You open YouTube on your phone (Client sends request)
2. Request travels through the internet (Network)
3. YouTube's server receives it (Server processes)
4. Server gets the video from storage (Database query)
5. Video is sent back to your phone (Response)

Memory Trick

Think of ordering pizza online:
* You = Client
* Pizza website = Server
* Order goes through internet = Network
* Pizza recipes stored = Database

Types of Architecture

1. ONE SERVER (Simple)
   [Client] --> [Single Server + Database]
   Good for: Small apps, personal projects

2. SEPARATE SERVER & DATABASE (Better)
   [Client] --> [Server] --> [Database]
   Good for: Medium apps

3. MULTIPLE SERVERS (Best for big apps)
   [Client] --> [Load Balancer] --> [Server 1]

--> [Server 2] --> [Database]
--> [Server 3]
Good for: Apps with millions of users

Real Example: WhatsApp Message

1. You type "Hi" and press send (Client)
2. Message goes to WhatsApp server (Server receives)
3. Server checks: Is your friend online? (Processing)
4. If yes: Send directly. If no: Store in database (Decision)
5. Friend’s phone gets the message (Delivery)

Today’s Quiz

1. In the restaurant analogy, what is the "waiter"?
2. Why would you need multiple servers?
3. Draw the journey of a WhatsApp message.

DAY 3: Networks & Protocols (How Computers Talk)

The Big Idea

Computers talk to each other using rules called protocols. It’s like how we have rules for talking on the phone:
* You say "Hello" first
* You wait for the other person to respond
* You say "Bye" before hanging up

The Main Protocols

HTTP/HTTPS (HyperText Transfer Protocol)
* Used by: Websites, Apps
* Like: Sending a letter and getting a reply
* Example: When you open google.com

TCP (Transmission Control Protocol)
* Used by: When you need ALL data to arrive
* Like: Sending a registered letter (you get confirmation)
* Example: Loading a webpage (every piece must arrive)

UDP (User Datagram Protocol)
* Used by: When speed matters more than perfection
* Like: Shouting in a crowd (some people might miss it)
* Example: Video calls, online gaming

Memory Trick

* TCP = Takes Careful Path (reliable, slow)
* UDP = Ultra Delivery Power (fast, might lose some)

IP Address

Every device on the internet has an address, like a home address.
* Example: 192.168.1.1
* It’s how computers find each other

Port Numbers

Think of an IP address as an apartment building, and port as the apartment number.
* Port 80: HTTP (websites)
* Port 443: HTTPS (secure websites)
* Port 3306: MySQL database

Full address: 192.168.1.1:443
[Building] :[Apartment]

DNS (Domain Name System)

Humans remember names, not numbers!
* You type: www.google.com
* DNS converts to: 142.250.80.46
* Like a phone book for the internet!

Today’s Quiz

1. What’s the difference between TCP and UDP?
2. What does DNS do?
3. Why does YouTube use UDP for video streaming?

DAY 4: APIs - How Apps Talk to Each Other

The Big Idea

An API (Application Programming Interface) is like a menu at a restaurant:
* The menu shows you what you CAN order
* You don’t need to know HOW the kitchen makes it
* You just ask for what you want, and you get it!

Real Example

When a weather app shows you temperature:
1. App asks Weather API: "What’s the temperature in New York?"
2. API replies: "72 degrees, sunny"
3. App shows it to you beautifully

The app doesn’t have a thermometer - it ASKS another service!

Types of APIs

REST API (Most Common)
* Uses HTTP methods like a library system:
* GET = Read a book (get data)
* POST = Add a new book (create data)
* PUT = Replace a book (update data)
* DELETE = Remove a book (delete data)

```
GET /users/123        --> Give me info about user 123
POST /users           --> Create a new user
PUT /users/123        --> Update user 123’s info
DELETE /users/123     --> Delete user 123
```

GraphQL
* Like ordering custom pizza: "I want only cheese and mushrooms"
* You ask for EXACTLY what you need, nothing more
* Facebook created it!

gRPC
* Super fast, like a bullet train
* Used when servers talk to each other
* Google created it!

Memory Trick

REST = Restaurant Style - You pick from the menu
GraphQL = Grocery Style - You pick exact ingredients
gRPC = Speed Style - Fastest communication

JSON - The Language of APIs

```
{
  "name": "John",
  "age": 14,
  "hobbies": ["gaming", "coding", "reading"]
}
```

JSON is how data is packaged and sent. Like putting a letter in an envelope with a specific format.

Today's Quiz

1. What HTTP method would you use to create a new post?
2. Why is GraphQL useful for mobile apps?
3. Write a JSON object for your favorite movie.

DAY 5: Databases - Where Data Lives

The Big Idea

A database is like a super organized filing cabinet that can find any file in milliseconds, even if there are billions of files!

Two Main Types

1. SQL Databases (Relational) - Like Excel Spreadsheets
* Data stored in tables with rows and columns
* Everything is organized and connected
* Examples: MySQL, PostgreSQL, Oracle

```
USERS TABLE:
| id  | name    | email           | age |
|-----|---------|-----------------|-----|
| 1   | Alice   | alice@mail.com  | 25  |
```

| 2 | Bob | bob@mail.com | 30 |

POSTS TABLE:

id	user_id	content	likes
1	1	"Hello World!"	50
2	2	"Love coding"	120

2. NoSQL Databases (Non-Relational) - Like a Storage Room

* Data stored in flexible formats
* No strict rules about structure
* Examples: MongoDB, Cassandra, Redis

```
{
  "id": 1,
  "name": "Alice",
  "posts": [
    {"content": "Hello World!", "likes": 50},
    {"content": "System Design is fun!", "likes": 200}
  ],
  "friends": ["Bob", "Charlie"]
}
```

When to Use What?

Use SQL When...	Use NoSQL When...
Data has clear structure	Data changes shape often
You need complex queries	You need super fast reads
Banking, e-commerce	Social media, gaming
Relationships matter	Scale is massive

Memory Trick

* SQL = Structured Query Language = Everything in neat boxes
* NoSQL = Not Only SQL = Flexible, like a backpack (throw anything in)

ACID Properties (SQL’s Superpowers)

* Atomicity: All or nothing (like a bank transfer - both accounts change or neither)
* Consistency: Rules are always followed
* Isolation: Transactions don’t mess with each other
* Durability: Once saved, it stays saved forever

Today's Quiz

1. Would you use SQL or NoSQL for a banking app? Why?
2. What does ACID stand for?
3. Give an example where NoSQL is better than SQL.

DAY 6: Scaling - Handling More Users

The Big Idea

Imagine your lemonade stand gets super popular. Yesterday 10 people came. Today 10,000 people want lemonade! What do you do?

Scaling = Making your system handle more users without breaking.

Two Ways to Scale

1. Vertical Scaling (Scale UP) - Get a BIGGER Machine
* Like replacing your bicycle with a motorcycle
* Buy a more powerful server (more RAM, CPU, storage)
* Simple but has limits (you can't make one machine infinitely powerful)

    Before: [Small Server - 4GB RAM]
    After: [HUGE Server - 256GB RAM]

2. Horizontal Scaling (Scale OUT) - Get MORE Machines
* Like hiring more lemonade sellers
* Add more servers that share the work
* Can scale almost infinitely!

    Before: [1 Server handling everything]
    After: [Server 1] [Server 2] [Server 3] [Server 4]
        All sharing the work!

Memory Trick

* Vertical = Think Very big (one giant machine going UP)
* Horizontal = Think Herd (many machines standing side by side)

Real Example: Netflix

* Netflix has 200+ million users
* They can't use one super computer
* They use THOUSANDS of servers (horizontal scaling)
* Servers are spread across the world

The Problem with Horizontal Scaling

If you have many servers, how does a user know which one to talk to?
Answer: Load Balancer (We'll learn this tomorrow!)

Comparison Table

Feature | Vertical Scaling | Horizontal Scaling
Cost | Expensive (big machines cost a lot) | Cheaper (many small machines)
Limit | Has a ceiling | Almost unlimited
Complexity | Simple | Complex (need coordination)
Downtime | Need to shut down to upgrade | Can add servers without stopping
Risk | Single point of failure | If one dies, others continue

Today's Quiz

1. Your app just got featured on TV and traffic jumped 100x. Which scaling do you choose? Why?
2. Why can't we just keep doing vertical scaling forever?
3. Name one company that uses horizontal scaling.

DAY 7: Load Balancers - The Traffic Police

The Big Idea

A Load Balancer is like a traffic police officer at a busy intersection. It directs cars (requests) to different roads (servers) so no single road gets too crowded.

Why We Need It

Without a load balancer:

```
10,000 requests --> [Server 1] --> CRASH! Too much!
```

With a load balancer:

10,000 requests --> [Load Balancer] --> [Server 1] 3,333 requests
--> [Server 2] 3,333 requests
--> [Server 3] 3,334 requests
Happy servers!

Load Balancing Algorithms

1. Round Robin (Take turns)
* Request 1 -> Server A
* Request 2 -> Server B
* Request 3 -> Server C
* Request 4 -> Server A (back to start)
* Like dealing cards in a card game!

2. Least Connections (Who's least busy?)
* Send to the server handling fewest requests right now
* Like choosing the shortest line at a grocery store

3. IP Hash (Same person, same server)
* Your IP address decides which server you go to
* Like having your own assigned desk at school

4. Weighted Round Robin (Stronger servers get more)
* Powerful server gets 5 requests, weak one gets 2
* Like giving more homework to the smartest student

Memory Trick

L.O.A.D = Levels Out All Demands

Real World Load Balancers

* Nginx - Very popular, free
* AWS ELB - Amazon's cloud load balancer
* HAProxy - High performance

Types of Load Balancers

Layer 4 (Transport): Looks at IP address and port
- Faster, simpler
- Like sorting mail by zip code

Layer 7 (Application): Looks at actual content
- Smarter, can route based on URL
- Like reading the letter to decide where it goes

Today’s Quiz

1. What load balancing algorithm is best for a gaming server?
2. What happens if the load balancer itself crashes?
3. Draw a system with 2 load balancers and 4 servers.

DAY 8: Caching - The Speed Superpower

The Big Idea

Caching is like keeping your favorite snacks on your desk instead of walking to the kitchen every time. The data you use most often is stored closer to you!

Why Caching Matters

* Database query: 100 milliseconds (slow)
* Cache lookup: 1 millisecond (100x faster!)

Real Life Caching

1. Your brain caches multiplication tables (you don’t calculate 7x8 each time)
2. Your browser caches website images (doesn’t re-download Google’s logo)
3. YouTube caches popular videos near you

Where to Cache?

```
[User] --> [Browser Cache] --> [CDN Cache] --> [App Server Cache] --> [Database Cache] --> [Database]
 Fastest                                                                                    Slowest
```

Cache Strategies

1. Cache-Aside (Lazy Loading)

    1. App asks cache: "Do you have this data?"
    2. Cache: "No" (Cache Miss)
    3. App asks database, gets data
    4. App stores data in cache for next time
    5. Next request: Cache says "YES!" (Cache Hit)

2. Write-Through

1. App writes data to cache AND database at same time
2. Always in sync, but slower writes

3. Write-Behind (Write-Back)

1. App writes to cache only (super fast!)
2. Cache writes to database later (in background)
3. Risk: If cache dies before writing, data is lost!

Memory Trick

CACHE = Commonly Accessed Content Held for Efficiency

Popular Cache Tools

* Redis - Most popular, stores data in memory
* Memcached - Simple and fast
* CDN (Content Delivery Network) - Caches content worldwide

Cache Problems

1. Cache Invalidation (When to update the cache?)
* "There are only two hard things in computer science: cache invalidation and naming things" - Phil Karlton

2. Cache Stampede (Everyone asks at once)
* Cache expires, 1000 users all hit database simultaneously!

Today’s Quiz

1. Why is cache faster than a database?
2. What’s a cache miss?
3. When would write-behind cache be dangerous?

DAY 9: CDN - Content Delivery Network

The Big Idea

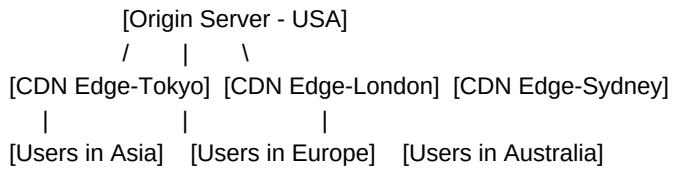
A CDN is like having copies of a popular book in every library in the world instead of just one library. When someone wants the book, they go to the nearest library!

The Problem CDN Solves

Without CDN:
* A user in Japan loads a website hosted in USA
* Data travels 10,000 km = SLOW (300ms delay)

With CDN:
* Website content is copied to a server in Japan
* User gets data from nearby server = FAST (20ms delay)

How CDN Works



What CDN Stores

* Images and videos
* CSS and JavaScript files
* Static HTML pages
* Downloads (apps, PDFs)
* Streaming content

Memory Trick

CDN = Copies Distributed Nearby

Real Examples

* Netflix uses CDN to stream movies (their CDN is called Open Connect)
* Instagram stores your photos on CDN
* Gaming - Game updates download from nearest CDN

Popular CDN Providers

* CloudFlare (also provides security)
* AWS CloudFront
* Akamai (one of the oldest)
* Google Cloud CDN

Push vs Pull CDN

Push CDN:
* YOU upload content to CDN manually
* Good for: Content that doesn't change often
* Like: Putting books on library shelves yourself

Pull CDN:
* CDN automatically grabs content when first requested
* Good for: Dynamic content
* Like: Library orders a book when someone first asks for it

Today's Quiz

1. Why is a CDN faster than a single server?
2. Would you use CDN for a banking transaction? Why or why not?
3. What's the difference between Push and Pull CDN?

DAY 10: Database Indexing - Finding Things Fast

The Big Idea

An index in a database is like the index at the back of a textbook. Without it, you'd have to read every single page to find what you want!

The Problem

Imagine a database with 1 BILLION users:
* Without index: Check all 1 billion rows = 10 minutes!
* With index: Jump directly to the right row = 0.001 seconds!

How Indexing Works

> Without Index (Full Table Scan):
> Page 1 -> Page 2 -> Page 3 -> ... -> Page 1,000,000 -> FOUND IT!
>
> With Index (B-Tree):
> Start -> Is it > M? Yes -> Is it > S? No -> Check P-S section -> FOUND "Smith"!
> (Only 3 steps instead of 1 million!)

Real Life Analogy

Finding "elephant" in a dictionary:
* Without index: Read every word from A to E... (hours!)
* With index: Go to "E" section, then "El"... (seconds!)

Types of Indexes

1. Primary Index (Main ID)
* Like a student roll number
* Every table should have one
* Usually the ID column

2. Secondary Index (Extra shortcuts)
* Like an index by subject in a library
* Example: Index on "email" column for fast login

3. Composite Index (Multiple columns)
* Like finding a book by BOTH author AND title
* Example: Index on (city, age) for queries like "all 25-year-olds in NYC"

Memory Trick

INDEX = Instant Navigational Data for EXtreme speed

The Trade-off

* Good: Reading is SUPER fast
* Bad: Writing is slightly slower (need to update the index too)
* Bad: Takes extra storage space

When to Create an Index

* Columns you search by often (WHERE clause)
* Columns you sort by (ORDER BY)
* Columns you join on

When NOT to Create an Index

* Small tables (full scan is fine)
* Columns that change constantly
* Columns with few unique values (like gender: M/F)

Today's Quiz

1. Why does an index make reads faster but writes slower?
2. Would you index a column with only 2 possible values?
3. What data structure do most indexes use?

DAY 11: Database Replication - Copies for Safety

The Big Idea

Database replication is like having backup copies of your homework - if you lose one, you still have others! It also lets multiple people read different copies at the same time.

Why Replicate?

1. Safety: If one database dies, others have the data
2. Speed: Users read from the nearest copy
3. Load: Spread reading across multiple copies

Master-Slave (Leader-Follower) Pattern

```
[Users Write] --> [Master DB] -- copies to --> [Slave 1] <-- [Users Read]
                                  --> [Slave 2] <-- [Users Read]
                                  --> [Slave 3] <-- [Users Read]
```

* Master: Handles ALL writes
* Slaves: Handle reads (copies of master)
* One master, many slaves

Master-Master Pattern

```
[Users] --> [Master 1] <-- syncs --> [Master 2] <-- [Users]
```

* Both can handle reads AND writes
* More complex (what if both write at the same time?)

Memory Trick

REPLICA = Redundant Extra Protection Locally In Case of Accidents

Replication Lag

* The time it takes for slave to get master's latest data
* Usually milliseconds, but can be seconds
* Problem: User writes to master, reads from slave, doesn't see their own update!

Solution: Read-Your-Own-Writes

* After a user writes, read from MASTER for that user
* Everyone else reads from slaves
* Best of both worlds!

Today's Quiz

1. Why can't all databases be masters?
2. What is replication lag?
3. If the master dies, what happens?

DAY 12: Database Sharding - Splitting Data

The Big Idea

Sharding is like splitting a giant phonebook into smaller books - one for A-F, one for G-L, etc. Each piece is easier to manage and search!

Why Shard?

* Your database has 10 BILLION rows
* One machine can't hold it all
* Solution: Split data across multiple machines!

How Sharding Works

```
All Users (10 billion)
        |
   [Shard Key: First letter of username]
        |
[Shard 1: A-F]  [Shard 2: G-L]  [Shard 3: M-R]  [Shard 4: S-Z]
 2.5 billion      2.5 billion      2.5 billion      2.5 billion
```

Sharding Strategies

1. Range-Based Sharding
* Split by ranges: Users 1-1M on Shard 1, 1M-2M on Shard 2
* Problem: Uneven distribution (what if most users have IDs 1-1M?)

2. Hash-Based Sharding
* Apply math formula to decide which shard
* hash(user_id) % number_of_shards = shard_number
* More even distribution!

3. Geographic Sharding
* US users on US shard, European users on EU shard
* Faster for users (data is nearby)

Memory Trick

SHARD = Split Horizontally Across Replicated Databases

Problems with Sharding

1. Joins are hard: Data on different machines can’t easily be combined
2. Rebalancing: Adding a new shard means moving data around
3. Hotspots: One shard might get way more traffic

Real Example

* Instagram shards by user_id
* Discord shards by server (guild) ID
* Uber shards by geographic location

Today’s Quiz

1. What’s the difference between sharding and replication?
2. Why is hash-based sharding more even than range-based?
3. A social media app has 500 million users. Design a sharding strategy.

DAY 13: Consistent Hashing - Smart Data Distribution

The Big Idea

Consistent hashing is like arranging people in a circle instead of a line. When someone leaves or joins, only their neighbors are affected, not everyone!

The Problem It Solves

Regular hashing: server = hash(key) % N
* If you add/remove a server (N changes), EVERYTHING moves!
* Like rearranging ALL lockers when one new locker is added.

Consistent hashing:
* Only ~1/N of the data moves when a server changes
* Like adding a new seat at a round table - only neighbors shift

How It Works

Imagine a clock (circle from 0 to 360 degrees):

```
        0/360
          |
    Server A (at 90)
          |
    330 --+-- 90
          |
    Server C (at 210)    Server B (at 150)
          |
        180
```

Data "photo_123" hashes to position 120 -> goes to Server B (next clockwise)
Data "video_456" hashes to position 200 -> goes to Server C (next clockwise)

When a Server Joins/Leaves

* Server B dies: Only B's data moves to next server (C)
* Other servers are NOT affected!
* Compare: Regular hashing would reshuffle everything

Memory Trick

Think of it as a CLOCK - data goes to the next server clockwise. Adding/removing a server only affects the slice next to it.

Virtual Nodes (Making it Even)

Problem: Servers might be unevenly spaced on the circle
Solution: Each real server gets multiple positions (virtual nodes)

Server A gets positions: 30, 120, 250
Server B gets positions: 70, 180, 330
Now the load is much more even!

Where It’s Used

* Amazon DynamoDB - Distributed database
* Apache Cassandra - NoSQL database
* Discord - Distributing chat servers
* Memcached - Distributed caching

Today’s Quiz

1. Why is consistent hashing better than regular hashing for distributed systems?
2. What are virtual nodes and why do we need them?
3. If we have 5 servers and add a 6th, how much data moves?

DAY 14: CAP Theorem - The Impossible Triangle

The Big Idea

The CAP Theorem says that in a distributed system, you can only have 2 out of 3 things. It’s like saying you can only pick 2 toppings on your pizza, never all 3!

The Three Properties

C - Consistency: Every read gets the LATEST data
* Like: Everyone sees the same scoreboard at the same time

A - Availability: System ALWAYS responds (never says "come back later")
* Like: The store is ALWAYS open, never closed

P - Partition Tolerance: System works even if network breaks between servers
* Like: The school still functions even if the intercom breaks

The Rule

In real distributed systems, network issues WILL happen (P is required).
So you really choose between:
* CP (Consistency + Partition Tolerance): Always correct data, might be unavailable sometimes

* AP (Availability + Partition Tolerance): Always available, might show slightly old data

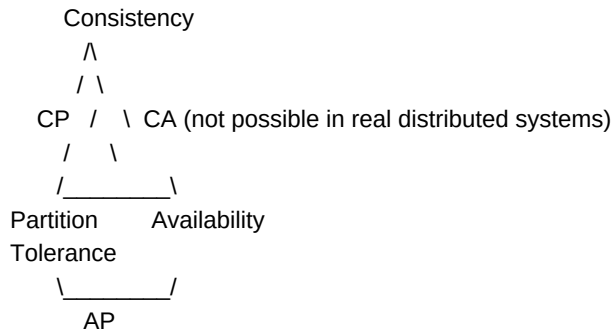
Memory Trick

CAP = Can’t Always Pick (all three)

Real World Examples

System | Choice | Why
Banks | CP | Money MUST be accurate (you’d rather wait than get wrong balance)
Social Media | AP | Better to show an old post than show nothing
Google Search | AP | Better to show some results than "Google is down"
Stock Trading | CP | Prices MUST be accurate
Shopping Cart | AP | Better to let users add items than say "try later"

Visual



Today’s Quiz

1. Why is Partition Tolerance usually required?
2. Would you choose CP or AP for a chat application?
3. Can a single-server system have all three? Why?

DAY 15: Message Queues - The Post Office of Systems

The Big Idea

A message queue is like a post office mailbox. The sender drops a letter and walks away. The receiver picks it up whenever they’re ready. They don’t need to be

available at the same time!

Why Message Queues?

Without queue (synchronous):

```
[User] --> [Server A] --> waits --> [Server B processes] --> response
User waits the ENTIRE time!
```

With queue (asynchronous):

```
[User] --> [Server A] --> drops message in queue --> "Done!" (immediate response)
                [Queue] --> [Server B picks up and processes later]
```

Real Examples

1. Email Sending
* You sign up for a website
* Server says "Welcome!" immediately
* Welcome email is queued and sent in background (you don’t wait for it)

2. Video Processing (YouTube)
* You upload a video
* YouTube says "Processing..." immediately
* Video encoding happens in background via queue

3. Food Delivery
* Order placed -> confirmation shown immediately
* Restaurant notified via queue
* Driver assigned via queue

Key Concepts

Producer: Sends messages (like you sending a letter)
Consumer: Receives and processes messages (like the recipient)
Queue: Stores messages in between (like the mailbox)

```
[Producer] --> [Queue: msg1, msg2, msg3] --> [Consumer]
```

Popular Message Queues

* RabbitMQ - Easy to use, reliable
* Apache Kafka - Super high-throughput, used by LinkedIn/Netflix
* Amazon SQS - Cloud-based, managed by AWS
* Redis - Can also work as a simple queue

Memory Trick

QUEUE = Quickly Ushers Updates for Eventual Execution

Benefits

1. Decoupling: Services don't depend on each other directly
2. Resilience: If consumer dies, messages wait in queue
3. Scalability: Add more consumers to process faster
4. Smoothing: Handle traffic spikes gracefully

Today's Quiz

1. Why would YouTube use a message queue for video uploads?
2. What happens if the consumer is down?
3. Name 3 scenarios where async processing is better than sync.

DAY 16: Microservices vs Monolith

The Big Idea

Imagine building with LEGO blocks vs carving from one big rock:
* Monolith = One big rock (everything together)
* Microservices = LEGO blocks (small pieces that fit together)

Monolith Architecture

```
[ONE BIG APPLICATION]
|-- User Management
|-- Payment Processing
|-- Email Service
|-- Search
|-- Notifications
|-- Analytics
ALL in ONE codebase, ONE deployment
```

Pros:
* Simple to develop initially
* Easy to test (everything is together)
* Simple deployment (one thing to deploy)

Cons:

* Gets messy as it grows (spaghetti code)
* One bug can crash EVERYTHING
* Can’t scale parts independently
* Team conflicts (everyone works on same code)

Microservices Architecture

[User Service] [Payment Service] [Email Service]
[Search Service] [Notification Service] [Analytics Service]
Each is independent! Each has its own database!

Pros:
* Scale each service independently
* Different teams own different services
* One crash doesn’t affect others
* Use best technology for each service

Cons:
* Complex communication between services
* Harder to debug across services
* Need infrastructure for service discovery
* Data consistency is harder

Memory Trick

* MONO = ONE (like monolingual = one language)
* MICRO = TINY (like microscope = seeing tiny things)

When to Use What?

Startup (< 5 engineers) | Big Company (100+ engineers)
Use Monolith | Use Microservices
Move fast, iterate | Scale independently
Simple deployment | Team autonomy

Real Examples

* Netflix: Started as monolith, migrated to 700+ microservices
* Amazon: Each team owns their own service
* Uber: Separate services for rides, payments, maps, matching

Today’s Quiz

1. Why did Netflix move from monolith to microservices?
2. For a brand new startup with 3 engineers, which would you recommend?

3. What’s the biggest challenge with microservices?

DAY 17: Rate Limiting - Preventing Abuse

The Big Idea

Rate limiting is like a bouncer at a club - only letting a certain number of people in per hour. It prevents any single user from overwhelming your system!

Why Rate Limiting?

* Prevent DDoS attacks (hackers flooding your system)
* Ensure fair usage (one user can’t hog everything)
* Control costs (API calls cost money)
* Protect backend services from overload

Common Algorithms

1. Token Bucket

    Bucket holds 10 tokens. Each request takes 1 token.
    Tokens refill at rate of 2 per second.
    - Request 1-10: Allowed (10 tokens used)
    - Request 11: DENIED (bucket empty)
    - Wait 1 second: 2 tokens refilled
    - Request 12-13: Allowed

Like: A candy machine that drops 2 candies per minute

2. Sliding Window

    Window = last 60 seconds
    Rule = Max 100 requests per minute

    Time 0:00 - 0:59: You made 100 requests (at limit!)
    Time 0:30 - 1:29: Count only requests in THIS window

Like: Counting cars on a highway in the last hour

3. Fixed Window

    Each minute resets the counter
    Minute 1: 0/100 used... 50/100... 100/100 -> BLOCKED
    Minute 2: Counter resets! 0/100 again

Memory Trick

RATE LIMIT = Restricting Access To Ensure Lasting Integrity & Managed Traffic

Where to Apply Rate Limiting

1. API Gateway - Before requests reach your servers
2. Per User - Each user gets their own limit
3. Per IP - Prevent one computer from flooding
4. Per Endpoint - Login page has stricter limits

Real Examples

* Twitter/X: 300 tweets per 3 hours
* GitHub API: 5000 requests per hour
* Google Maps: 25,000 map loads per day (free tier)

HTTP Response When Rate Limited

```
HTTP 429 - Too Many Requests
Headers:
  X-RateLimit-Limit: 100
  X-RateLimit-Remaining: 0
  Retry-After: 30 seconds
```

Today’s Quiz

1. What HTTP status code means "rate limited"?
2. Why would you rate limit a login endpoint more strictly?
3. Design a rate limiter for an API that allows 1000 requests/minute.

DAY 18: Proxies - The Middlemen

The Big Idea

A proxy is like a personal assistant who talks to people on your behalf. You tell the assistant what you need, and they go get it for you.

Forward Proxy (Client-side)

Sits between users and the internet. Protects the CLIENT.

```
[User] --> [Forward Proxy] --> [Internet/Servers]
```

Use cases:
* Hide your IP address (privacy)
* Access blocked websites (school/work)
* Cache frequently accessed content
* Filter content (parental controls)

Reverse Proxy (Server-side)

Sits between internet and servers. Protects the SERVER.

```
[Internet/Users] --> [Reverse Proxy] --> [Server 1]
                                     --> [Server 2]
                                     --> [Server 3]
```

Use cases:
* Load balancing (distribute traffic)
* SSL termination (handle encryption)
* Caching (serve cached content fast)
* Security (hide server details)

Memory Trick

* Forward proxy = For the client (Forward = Front of client)
* Reverse proxy = Rear guard for servers (Reverse = Behind, protecting servers)

Real Examples

* Nginx - Most popular reverse proxy
* VPN - Type of forward proxy
* CloudFlare - Reverse proxy + CDN + security
* Corporate networks - Forward proxy to filter employee internet

Today’s Quiz

1. What’s the difference between forward and reverse proxy?
2. When you use a VPN, what type of proxy is it?
3. Why does Netflix use a reverse proxy?

DAY 19: Latency vs Throughput

The Big Idea

* Latency = How FAST one thing gets done (speed of one car)
* Throughput = How MUCH gets done in a period (cars per hour on highway)

Analogy: Water Pipes

* Latency = How quickly the first drop reaches the other end
* Throughput = How much water flows per second (depends on pipe width)

A thin pipe can have low latency (short pipe) but low throughput.
A wide pipe can have high throughput but maybe high latency (long pipe).

Numbers Every Engineer Should Know

Operation | Time
L1 cache reference | 0.5 nanoseconds
L2 cache reference | 7 nanoseconds
RAM reference | 100 nanoseconds
SSD read | 150 microseconds
Hard disk read | 10 milliseconds
Send packet US to Europe | 150 milliseconds
Read 1 MB from SSD | 1 millisecond
Read 1 MB from disk | 20 milliseconds
Read 1 MB from network | 10 milliseconds

Memory Trick

Latency = Late (how late is the response?)
Throughput = Through (how much goes through?)

How to Improve Latency

1. Cache data closer to user
2. Use CDN
3. Reduce number of network hops
4. Use faster storage (SSD instead of HDD)
5. Compress data before sending

How to Improve Throughput

1. Add more servers (horizontal scaling)
2. Use wider network bandwidth
3. Process things in parallel (batching)
4. Use message queues for async processing
5. Optimize database queries

Real Example

Google Search:
* Latency goal: < 200ms per search
* Throughput: 8.5 billion searches per day = ~100,000 per second

Today’s Quiz

1. Can you have low latency but low throughput?
2. What’s more important for a video game: latency or throughput?
3. What’s more important for file downloads: latency or throughput?

DAY 20: System Design Framework (How to Approach ANY Problem)

The Big Idea

When someone asks you to "Design Twitter" or "Design Uber", don’t panic! Follow this 4-step framework every single time.

The STAR Framework for System Design

S - Scope (3-5 minutes)
* What features do we need?
* How many users?
* What are the constraints?
* Ask clarifying questions!

T - Think High Level (5-10 minutes)
* Draw the big picture
* Identify main components
* Show data flow

A - Architect in Detail (15-20 minutes)
* Database schema
* API design
* Deep dive into key components
* Discuss trade-offs

R - Review & Resolve (5 minutes)
* Identify bottlenecks
* Discuss scalability

* Handle edge cases
* Mention monitoring/alerting

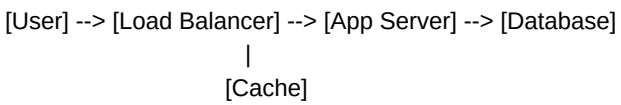
Memory Trick

STAR = Scope, Think, Architect, Review
(Like getting a gold STAR for your design!)

Example: "Design a URL Shortener"

S - Scope:
* How many URLs per day? 100 million
* How long to keep them? 5 years
* Custom short URLs? Yes
* Analytics needed? Click count

T - Think High Level:



A - Architect:
* Short URL generation: Base62 encoding
* Database: NoSQL (simple key-value)
* Cache: Redis for hot URLs
* Read-heavy system (100:1 read/write ratio)

R - Review:
* Bottleneck: Database for popular URLs -> Cache solves this
* Scale: Shard by first character of short URL
* Edge case: What if two people want same custom URL?

Week 1-3 Summary Checklist

* [ ] Client-Server architecture
* [ ] Networks & Protocols
* [ ] APIs (REST, GraphQL, gRPC)
* [ ] Databases (SQL vs NoSQL)
* [ ] Scaling (Vertical vs Horizontal)
* [ ] Load Balancers
* [ ] Caching
* [ ] CDN
* [ ] Indexing
* [ ] Replication & Sharding
* [ ] Consistent Hashing

* [ ] CAP Theorem
* [ ] Message Queues
* [ ] Microservices
* [ ] Rate Limiting
* [ ] Proxies
* [ ] Latency vs Throughput
* [ ] System Design Framework

MINI PROJECT: Design a Simple Chat Application

Design a basic chat app (like WhatsApp) with:
* 1-on-1 messaging
* Group chat (up to 100 people)
* Online/offline status
* Message delivery status (sent, delivered, read)

Draw the architecture and identify which concepts from Days 1-20 you'd use!

PHASE 2: CORE CONCEPTS (Days 21-40)

Deepening Your Knowledge

DAY 21: Availability & Reliability

The Big Idea

* Availability = Is the system UP right now? (Can I use it?)
* Reliability = Does the system work CORRECTLY over time? (Can I trust it?)

A system can be available but unreliable (website is up but shows wrong data).

The Nines of Availability

Availability	Downtime/Year	Called
99%	3.65 days	Two nines
99.9%	8.76 hours	Three nines
99.99%	52.6 minutes	Four nines
99.999%	5.26 minutes	Five nines
99.9999%	31.5 seconds	Six nines

Memory Trick

Each nine = 10x less downtime
* Google targets: 99.99% (52 min downtime/year)
* AWS targets: 99.99% for most services
* A pacemaker: 99.9999% (life depends on it!)

How to Achieve High Availability

1. Redundancy: Multiple copies of everything
2. Failover: Automatic switch to backup
3. Health Checks: Constantly monitor if things are working
4. No Single Point of Failure (SPOF): If ONE thing breaking kills everything, that's bad!

Single Point of Failure (SPOF)

```
BAD:  [Users] --> [ONE Server] --> [ONE Database]
      If server dies = EVERYTHING dies!

GOOD: [Users] --> [Load Balancer] --> [Server 1] --> [DB Primary]
                  [LB Backup]     --> [Server 2] --> [DB Replica]
                                  --> [Server 3] --> [DB Replica]
      Multiple failures needed to bring system down!
```

Failover Types

* Active-Passive: One works, one waits (like a backup generator)
* Active-Active: Both work, share load (like two doors to a store)

Today's Quiz

1. What's the downtime per year for 99.9% availability?
2. Find the SPOF in this design: Users -> Server -> Database
3. Which is better: Active-Active or Active-Passive? Why?

DAY 22: Data Consistency Patterns

The Big Idea

When you have multiple copies of data, how do you keep them in sync?
Like: If you update your profile photo, when do all your friends see the new one?

Consistency Levels

1. Strong Consistency
* Everyone sees the same data at the same time
* Like: Bank balance (must be exact!)
* Slow but accurate
* Example: After transferring money, balance is immediately correct everywhere

2. Eventual Consistency
* Data will be the same EVENTUALLY (might take seconds)
* Like: Social media likes (it's okay if count is slightly off for a moment)
* Fast but temporarily inaccurate
* Example: You post a photo, friends see it a few seconds later

3. Causal Consistency
* Related events appear in correct order
* Like: In a chat, replies always appear after the original message
* Middle ground

Memory Trick

* Strong = Synchronized Instantly
* Eventual = Eventually Everyone sees it
* Causal = Cause before effect

Real World Choices

System | Consistency | Why
Banking | Strong | Money must be exact
Instagram Likes | Eventual | Who cares if it's 999 vs 1000 for 2 seconds?
Google Docs | Strong | Multiple editors must see same thing
DNS | Eventual | OK if new website takes hours to propagate
Stock prices | Strong | Traders need exact prices

Today's Quiz

1. Why is strong consistency slower?

2. Would a shopping cart use strong or eventual consistency?
3. Give an example where eventual consistency is dangerous.

DAY 23: Hashing & Checksums

The Big Idea

Hashing is like a fingerprint for data. You take any data (a file, password, message) and create a fixed-size unique identifier.

Properties of a Good Hash

1. Same input = Same output (always)
2. Different input = Different output (usually)
3. Can’t reverse it (can’t get original from hash)
4. Small change = Completely different hash

Example

```
hash("Hello") = "2cf24dba5fb0a30e26e83b2ac5b9e29e1b161e5c..."
hash("hello") = "b6ef26b11e18305c6c4aa87e5bfe7e0a2818c5f3..."
hash("Hello!") = "9b71d224bd62f3785d96d46ad3ea3d73319bfbc2..."
```

Notice: Tiny changes create completely different hashes!

Use Cases

1. Password Storage
* NEVER store passwords as plain text!
* Store hash instead: hash("MyPassword123") = "a1b2c3d4..."
* To verify: hash the input and compare hashes

2. Data Integrity
* Download a file + its hash
* Hash the downloaded file
* Compare: If hashes match, file is not corrupted!

3. Hash Tables (for fast data lookup)
* hash("user_123") tells you which bucket/server to look in

4. Deduplication

* Hash all files. Same hash? It’s a duplicate!

Common Hash Algorithms

* MD5 - Fast but not secure (don’t use for passwords)
* SHA-256 - Secure, used in Bitcoin
* bcrypt - Best for passwords (intentionally slow)

Memory Trick

HASH = Has A Specific Handle (unique identifier for anything)

Today’s Quiz

1. Why should you NEVER store plain text passwords?
2. What happens if two different inputs produce the same hash? (hint: collision)
3. Why is bcrypt "intentionally slow"?

DAY 24: WebSockets & Real-Time Communication

The Big Idea

Normal HTTP is like sending letters - you send a request, get a response, connection closes.
WebSockets are like a phone call - once connected, both sides can talk anytime!

HTTP vs WebSocket

HTTP (Half-duplex):
Client: "Any new messages?" --> Server: "No"
Client: "Any new messages?" --> Server: "No"
Client: "Any new messages?" --> Server: "Yes! Here it is"
(Client keeps asking... wasteful!)

WebSocket (Full-duplex):
Client: "Let’s stay connected" --> Server: "OK, connection open!"
Server: "Hey! New message for you!" (anytime, without asking)
Client: "Thanks! Here’s my reply" (anytime too)

When to Use WebSockets

* Chat applications (WhatsApp, Slack)
* Live sports scores
* Stock ticker (real-time prices)
* Online gaming (multiplayer)
* Live collaborative editing (Google Docs)
* Notifications

When NOT to Use WebSockets

* Simple CRUD apps (blog, e-commerce catalog)
* One-time data fetches
* File uploads

Other Real-Time Options

1. Polling (Checking repeatedly)

    Every 5 seconds: "Anything new?" "Anything new?" "Anything new?"
    Simple but wasteful!

2. Long Polling (Patient waiting)

    Client: "Tell me when something happens"
    Server: [waits... waits... waits... 30 seconds]
    Server: "Here's an update!"
    Client: Immediately asks again

3. Server-Sent Events (SSE)

    One-way: Server --> Client (only server pushes)
    Good for: Notifications, news feeds
    Can't send from client!

Memory Trick

* HTTP = Hello, here's my question, goodbye
* WebSocket = We're Staying connected

Today's Quiz

1. Why would an online game use WebSockets instead of HTTP?
2. What's the disadvantage of polling?
3. When would you choose SSE over WebSockets?

DAY 25: Authentication & Authorization

The Big Idea

* Authentication (AuthN) = WHO are you? (Proving your identity)
* Authorization (AuthZ) = WHAT can you do? (Permissions)

Like entering a company building:
* Authentication = Showing your ID badge (proving who you are)
* Authorization = Your badge only opens certain doors (what you're allowed to access)

Authentication Methods

1. Password-Based
* Username + Password
* Simple but can be stolen
* Always hash passwords!

2. Token-Based (JWT)

1. User logs in with username/password
2. Server creates a token (like a wristband at a concert)
3. User sends token with every request
4. Server verifies token (no need to check database each time!)

3. OAuth 2.0 (Login with Google/Facebook)

1. User clicks "Login with Google"
2. Google asks: "Allow this app to see your info?"
3. User says "Yes"
4. Google gives app a token
5. App uses token to get user's info from Google

4. Multi-Factor Authentication (MFA)
* Something you KNOW (password)
* Something you HAVE (phone with OTP)
* Something you ARE (fingerprint)

JWT (JSON Web Token) - Deep Dive

Header.Payload.Signature
eyJhbGciOiJIUzI1NiJ9.eyJ1c2VyX2lkIjoxMjN9.abc123signature

Header: {"alg": "HS256"} (algorithm used)
Payload: {"user_id": 123, "role": "admin", "exp": 1234567890}
Signature: Proves the token hasn't been tampered with

Memory Trick

* AuthN = "Are you Named who you say?" (Identity)
* AuthZ = "Are you Zoned for this?" (Permission)

Session vs Token

Sessions | Tokens (JWT)
Stored on server | Stored on client
Server remembers you | Server is stateless
Hard to scale | Easy to scale
Server uses memory | No server memory needed

Today's Quiz

1. What's the difference between authentication and authorization?
2. Why is JWT good for microservices?
3. What are the 3 factors in multi-factor authentication?

DAY 26: Logging, Monitoring & Alerting

The Big Idea

If your system is a car, then:
* Logging = The dashboard cam (records everything that happens)
* Monitoring = The dashboard gauges (speed, fuel, temperature)
* Alerting = The warning lights (tells you when something's wrong)

Logging

What to log:
* Errors and exceptions
* User actions (login, purchase)
* Performance data (response times)
* Security events (failed logins)

Log Levels:

DEBUG: Very detailed, for development only
INFO: Normal operations ("User 123 logged in")
WARN: Something unusual but not broken ("Disk 80% full")
ERROR: Something failed ("Payment failed for user 123")

FATAL: System is crashing ("Database connection lost")

Monitoring

Key Metrics (The 4 Golden Signals):
1. Latency - How long requests take
2. Traffic - How many requests per second
3. Errors - How many requests fail
4. Saturation - How full is your system (CPU, memory, disk)

Alerting Rules

IF response_time > 2 seconds for 5 minutes THEN alert
IF error_rate > 5% for 1 minute THEN page on-call engineer
IF disk_usage > 90% THEN warn
IF server is down for 30 seconds THEN critical alert

Memory Trick

LMA = Logs Monitor Alerts (in that order)
* First: LOG everything
* Then: MONITOR the patterns
* Finally: ALERT when something’s wrong

Tools

* Logging: ELK Stack (Elasticsearch + Logstash + Kibana), Splunk
* Monitoring: Prometheus, Grafana, Datadog
* Alerting: PagerDuty, OpsGenie

Today’s Quiz

1. What are the 4 Golden Signals?
2. What log level would "Database connection slow" be?
3. Design an alerting rule for a payment system.

DAY 27: Storage Systems - Blob, File, Block

The Big Idea

Different types of data need different types of storage, just like you store clothes in a closet, food in a fridge, and books on a shelf!

Types of Storage

1. Block Storage (Like a hard disk)
* Raw chunks of data
* Fastest, lowest level
* Used for: OS drives, databases
* Examples: AWS EBS, SAN

2. File Storage (Like a file cabinet)
* Organized in folders and files
* Has a path: /home/photos/vacation.jpg
* Used for: Shared drives, documents
* Examples: NFS, AWS EFS

3. Object/Blob Storage (Like a warehouse)
* Flat storage, each object has a unique key
* Unlimited scale
* Used for: Images, videos, backups, logs
* Examples: AWS S3, Google Cloud Storage

Comparison

Feature	Block	File	Object
Speed	Fastest	Medium	Slowest
Scale	Limited	Medium	Unlimited
Cost	Expensive	Medium	Cheapest
Best for	Databases	Shared files	Media, backups
Structure	Raw blocks	Hierarchy	Flat (key-value)

Memory Trick

* Block = Basic building blocks (raw, fast)
* File = Folders and files (organized)
* Object = Ocean of objects (unlimited, flat)

Real Example: Instagram

* Profile data: Block storage (database)
* Application code: File storage
* Photos and videos: Object storage (S3)
* Thumbnails: Object storage + CDN

Today’s Quiz

1. Where would Netflix store its movies?
2. Why is object storage cheapest?
3. Can you edit part of an object in blob storage?

DAY 28: Content Delivery & Streaming

The Big Idea

Streaming is like drinking from a water fountain - you consume as the water flows.
Downloading is like filling a bottle first, then drinking.

Streaming vs Downloading

```
Downloading: [Full video file transfers] ----> [Then you watch]
Streaming:   [Small chunks transfer] --> [Watch immediately while more loads]
```

How Video Streaming Works (Netflix/YouTube)

1. Video is split into small chunks (2-10 seconds each)
2. Each chunk is encoded in different qualities (360p, 720p, 1080p, 4K)
3. Your player requests chunks one by one
4. If internet is slow -> switch to lower quality
5. This is called Adaptive Bitrate Streaming

Protocols for Streaming

* HLS (HTTP Live Streaming) - Apple’s standard
* DASH (Dynamic Adaptive Streaming over HTTP) - Open standard
* WebRTC - Real-time (video calls)

CDN + Streaming

```
[Origin: Full video] --> Distributed to CDN edges worldwide
User in Tokyo gets video from Tokyo edge (not US origin)
Result: Fast loading, less buffering!
```

Memory Trick

STREAM = Serves Tiny Rounds of Entertainment And Media

Video Processing Pipeline (YouTube)

```
Upload --> [Transcoding: convert to multiple formats]
    --> [Quality: 360p, 720p, 1080p, 4K]
    --> [Chunking: split into 5-sec pieces]
    --> [Store in Object Storage]
    --> [Distribute to CDN]
    --> [User streams from nearest CDN]
```

Today's Quiz

1. Why does video quality change while watching Netflix?
2. What's the benefit of splitting video into chunks?
3. Why is WebRTC used for video calls but not Netflix?

DAY 29: Distributed Systems Fundamentals

The Big Idea

A distributed system is a group of computers working together that appears as one system to the user. Like a restaurant chain - you go to any location and get the same food, but behind the scenes, many kitchens are coordinating!

Why Distributed Systems?

1. No single computer is powerful enough
2. Users are spread around the world
3. Need to survive failures
4. Need to handle massive scale

Key Challenges

1. Network Failures
* Computers communicate over networks
* Networks can be slow, drop messages, or die
* You MUST design for network failure

2. Clock Synchronization
* Different computers have slightly different clocks
* "What time did this happen?" is surprisingly hard

* Solution: Logical clocks (Lamport timestamps)

3. Split Brain
* Network splits into two groups
* Each group thinks the other is dead
* Both might try to be "master" -> conflicts!

Consensus Problem

How do multiple computers agree on something?
Like: 5 friends trying to decide where to eat, but they can only text (and texts might not deliver)

Popular Solution: Raft Algorithm
* One node is elected "leader"
* Leader makes decisions
* Others follow the leader
* If leader dies, election happens!

Memory Trick

DISTRIBUTED = Dozens of Independent Systems Together Running In Better Unison Than Expected, Delivering

The Two Generals Problem

> General A wants to attack with General B.
> They send messengers through enemy territory.
> Problem: How do they BOTH know the other got the message?
> Surprise: This is IMPOSSIBLE to solve perfectly!

This is why distributed systems are fundamentally hard.

Today’s Quiz

1. What is the "split brain" problem?
2. Why can’t we just use wall clocks in distributed systems?
3. How does the Raft algorithm handle leader failure?

DAY 30: Leader Election & Coordination

The Big Idea

In a group of servers, sometimes ONE server needs to be the "boss" (leader). Leader election is the process of choosing that boss - fairly and automatically!

Why Do We Need a Leader?

* Someone needs to make final decisions
* Someone coordinates writes
* Someone assigns tasks to workers
* Avoid conflicts (two servers doing the same job)

How Leader Election Works

1. Bully Algorithm

    All servers have IDs: [1, 2, 3, 4, 5]
    - Highest living ID = Leader
    - Server 5 is leader
    - Server 5 dies
    - Servers 1-4 notice
    - Server 4 says "I'm the new leader!" (highest alive)
    - Like the tallest person in the room is captain

2. Raft Consensus

    - Servers start as "followers"
    - If no heartbeat from leader for X seconds...
    - A follower becomes "candidate" and asks for votes
    - If majority votes yes -> New leader!
    - Like a class electing a president

ZooKeeper - The Coordination King

ZooKeeper is like a shared notebook that all servers trust:
* Who is the current leader? (Check the notebook)
* Which servers are alive? (They sign in)
* What's the current configuration? (Written in notebook)

```
[Server 1] --\
[Server 2] ----> [ZooKeeper] <-- Source of truth
[Server 3] --/
```

Memory Trick

Leader = The One Everyone Agrees To Listen To
* Not the strongest, not the smartest

* Just the one everyone AGREES on!

Real Examples

* Kafka: Uses ZooKeeper for leader election of partitions
* Elasticsearch: Elects a master node
* etcd: Used by Kubernetes for coordination

Today’s Quiz

1. What happens if the leader dies?
2. Why do we need consensus for leader election?
3. What is ZooKeeper used for?

DAY 31: Event-Driven Architecture

The Big Idea

Instead of services ASKING each other for updates, they ANNOUNCE events and whoever cares listens. Like a school PA system - the announcement goes out, and only relevant people act on it!

Request-Driven vs Event-Driven

Request-Driven (Traditional):
Order Service: "Hey Inventory, reduce stock!"
Order Service: "Hey Email, send confirmation!"
Order Service: "Hey Analytics, log this!"
(Order service knows about everyone - tightly coupled)

Event-Driven:
Order Service: "ORDER_PLACED!" (announces to everyone)
Inventory: "I heard! Reducing stock."
Email: "I heard! Sending confirmation."
Analytics: "I heard! Logging it."
(Order service doesn’t know or care who listens!)

Key Concepts

Event: Something that happened ("user_signed_up", "order_placed")
Producer: Creates events (publishes)

Consumer: Listens for events (subscribes)
Event Bus/Broker: The channel events flow through

Event Patterns

1. Pub/Sub (Publish-Subscribe)

```
[Producer] --> [Topic: "orders"] --> [Consumer 1: Email]
                                 --> [Consumer 2: Inventory]
                                 --> [Consumer 3: Analytics]
All consumers get the same event!
```

2. Event Sourcing
* Store ALL events, not just current state
* Like a bank statement (list of transactions, not just balance)
* Can replay events to rebuild state!

```
Event Log:
1. Account Created (balance: 0)
2. Deposit $100 (balance: 100)
3. Withdraw $30 (balance: 70)
4. Deposit $50 (balance: 120)
Current state = replay all events!
```

Memory Trick

EVENT = Everything Ventures Effortlessly, Notifying Things

Benefits

1. Loose coupling: Services don’t know about each other
2. Scalability: Add new consumers without changing producers
3. Audit trail: All events are recorded
4. Replay: Can reprocess past events

Today’s Quiz

1. Why is event-driven architecture more scalable?
2. What’s the difference between Pub/Sub and a message queue?
3. How would you use event sourcing for a bank account?

DAY 32: Search Systems & Indexing

The Big Idea

How does Google search billions of web pages in 0.5 seconds? The secret is inverted indexes - like the index at the back of a book, but for the entire internet!

How Search Works

1. Crawling - Discover pages (like exploring every street in a city)
2. Indexing - Organize what you found (like creating a card catalog)
3. Ranking - Decide what's most relevant (like sorting best to worst)

Inverted Index

Normal: Document -> Words

```
Doc 1: "The cat sat on the mat"
Doc 2: "The dog sat on the log"
```

Inverted: Word -> Documents

```
"cat" -> [Doc 1]
"sat" -> [Doc 1, Doc 2]
"dog" -> [Doc 2]
"the" -> [Doc 1, Doc 2]
```

Now searching for "cat" instantly gives you Doc 1!

Elasticsearch - The Search Engine

* Built on Apache Lucene
* Distributed (spreads across many servers)
* Near real-time search
* Used by: Wikipedia, GitHub, Netflix

How to Design a Search System

```
[User types query] --> [Query Parser]
                            |
                       [Inverted Index]
                            |
                       [Ranking Algorithm]
                            |
                       [Return Top Results]
```

Memory Trick

Think of search like a library:
* Without index: Walk through every shelf, check every book
* With index: Look up the card catalog, go directly to the book

Relevance Ranking (TF-IDF)

* TF (Term Frequency): How often the word appears in this document
* IDF (Inverse Document Frequency): How rare the word is across all documents
* Rare words in a document = more relevant!

Today's Quiz

1. What is an inverted index?
2. Why is "the" a bad search term?
3. How would you design search for an e-commerce site?

DAY 33: Data Partitioning Strategies

The Big Idea

When your data is too big for one machine, you split it across multiple machines. The question is HOW to split it smartly!

Partitioning Strategies

1. Horizontal Partitioning (Sharding)
* Split rows across machines
* Each machine has ALL columns but SOME rows

    Machine 1: Users A-M
    Machine 2: Users N-Z

2. Vertical Partitioning
* Split columns across machines
* Each machine has ALL rows but SOME columns

    Machine 1: user_id, name, email (frequently accessed)
    Machine 2: user_id, bio, preferences (rarely accessed)

3. Functional Partitioning
* Split by function/feature

* Each machine handles a different type of data

    Machine 1: User data
    Machine 2: Product data
    Machine 3: Order data

Choosing a Partition Key

The partition key determines WHERE data goes. Choose wisely!

Good Partition Key:
* Evenly distributes data
* Queries mostly hit ONE partition
* Example: user_id (each user's data on one machine)

Bad Partition Key:
* Creates hotspots (uneven distribution)
* Example: country (US might have 70% of users)
* Example: date (today's partition gets ALL traffic)

Memory Trick

Partition = Pizza slicing
* Horizontal = Cut into rows (like cutting a pizza in strips)
* Vertical = Cut into columns (like cutting pizza in wedges)
* Functional = Different pizzas for different tables

Hotspot Problem & Solutions

Problem: One partition gets way more traffic
Solutions:
1. Add randomness to key (append random digit)
2. Split hot partitions further
3. Use consistent hashing with virtual nodes

Today's Quiz

1. What's the difference between horizontal and vertical partitioning?
2. Why is "date" often a bad partition key?
3. Design a partition strategy for Twitter's tweets.

DAY 34: Batch vs Stream Processing

The Big Idea

* Batch Processing = Washing clothes once a week (collect all, process together)
* Stream Processing = Assembly line at a factory (process each item as it arrives)

Batch Processing

Collect data --> Wait until there's a lot --> Process all at once

* Example: Netflix generating "recommended for you" overnight
* Example: Monthly bank statements
* Example: Daily sales reports

Tools: Hadoop, Apache Spark, AWS Batch

Stream Processing

Data arrives --> Process immediately --> Results in real-time

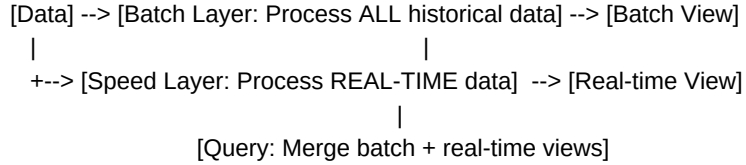
* Example: Fraud detection (block suspicious transaction instantly)
* Example: Live traffic updates on Google Maps
* Example: Real-time sports scores

Tools: Apache Kafka Streams, Apache Flink, Apache Storm

Comparison

Feature	Batch	Stream
When processed	Later (scheduled)	Immediately
Latency	Minutes to hours	Milliseconds to seconds
Data size	Very large	One event at a time
Complexity	Simpler	More complex
Use case	Reports, ML training	Alerts, real-time dashboards

Lambda Architecture (Best of Both!)



Memory Trick

* Batch = Big pile, process later (like laundry day)
* Stream = Straight through, no waiting (like conveyor belt)

Today's Quiz

1. Should fraud detection use batch or stream processing?
2. Why might you need BOTH batch and stream?
3. Name 3 real-world stream processing use cases.

DAY 35: API Gateway Pattern

The Big Idea

An API Gateway is like the front desk of a hotel. Guests don't go directly to housekeeping, restaurant, or maintenance. They tell the front desk what they need, and the front desk routes them!

Without API Gateway

```
Mobile App must know about:
- User Service (port 3001)
- Payment Service (port 3002)
- Order Service (port 3003)
- Notification Service (port 3004)
Complex! What if services move?
```

With API Gateway

```
Mobile App --> [API Gateway] --> User Service
                             --> Payment Service
                             --> Order Service
                             --> Notification Service
App only knows ONE address!
```

What API Gateway Does

1. Routing: Directs requests to correct service
2. Authentication: Checks if user is logged in
3. Rate Limiting: Prevents abuse

4. Load Balancing: Spreads traffic
5. Caching: Returns cached responses
6. Request/Response Transformation: Converts formats
7. Monitoring: Tracks all API calls

Memory Trick

API Gateway = AIRPORT
* Single entry point (front door)
* Security check (authentication)
* Directs you to correct gate (routing)
* Controls flow of people (rate limiting)

Popular API Gateways

* Kong - Open source, very popular
* AWS API Gateway - Managed by Amazon
* Nginx - Can act as API gateway
* Zuul - Netflix's gateway

BFF Pattern (Backend for Frontend)

Different clients need different data:

```
[Mobile App] --> [Mobile BFF] --> [Microservices]
[Web App]    --> [Web BFF]    --> [Microservices]
[TV App]     --> [TV BFF]     --> [Microservices]
```

Each BFF is customized for its client!

Today's Quiz

1. What are 3 responsibilities of an API Gateway?
2. Why would a mobile app need different data than a web app?
3. What happens if the API Gateway goes down?

DAY 36: Database Types Deep Dive

The Big Idea

Different databases are like different vehicles - a bus, car, motorcycle, and bicycle. Each is best for different situations!

Database Types

1. Relational (SQL)
* Tables with rows and columns
* Best for: Structured data, complex queries
* Examples: PostgreSQL, MySQL, Oracle
* Use when: Banking, e-commerce, ERP systems

2. Document (NoSQL)
* JSON-like documents
* Best for: Flexible schemas, rapid development
* Examples: MongoDB, CouchDB
* Use when: User profiles, content management, catalogs

3. Key-Value
* Simple: key -> value pairs
* Best for: Caching, sessions, simple lookups
* Examples: Redis, DynamoDB, Memcached
* Use when: Shopping carts, user sessions, leaderboards

4. Wide-Column
* Tables but columns can vary per row
* Best for: Time-series, IoT data, huge datasets
* Examples: Cassandra, HBase, ScyllaDB
* Use when: Sensor data, log analysis, messaging

5. Graph
* Nodes and relationships
* Best for: Connected data, social networks
* Examples: Neo4j, Amazon Neptune
* Use when: Social graphs, recommendation engines, fraud detection

6. Time-Series
* Optimized for timestamp-based data
* Best for: Metrics, monitoring, IoT
* Examples: InfluxDB, TimescaleDB, Prometheus

Decision Matrix

Need | Best Database Type
Complex queries with joins | SQL (PostgreSQL)
Flexible JSON documents | Document (MongoDB)
Lightning-fast lookups | Key-Value (Redis)
Social connections | Graph (Neo4j)
Metrics over time | Time-Series (InfluxDB)
Massive write throughput | Wide-Column (Cassandra)

Memory Trick

Pick database like picking a vehicle:
* Need precision? SQL (luxury car)
* Need flexibility? Document (SUV)
* Need speed? Key-Value (sports car)
* Need connections? Graph (subway map)

Today's Quiz

1. What database would you choose for a recommendation engine?
2. Why would you use a time-series database for server monitoring?
3. Can you use multiple database types in one system?

DAY 37: Idempotency & Retry Mechanisms

The Big Idea

Idempotent means doing something multiple times has the SAME effect as doing it once.
Like pressing an elevator button - pressing it 5 times doesn't make the elevator come
5 times faster!

Why Idempotency Matters

> Scenario: You click "Pay $100" button
> Network glitch: Your phone doesn't get the response
> Phone retries: Sends payment request again
> WITHOUT idempotency: You get charged $200!
> WITH idempotency: You get charged $100 (second request is ignored)

Idempotent vs Non-Idempotent

Operation | Idempotent? | Why
GET /users/123 | Yes | Reading doesn't change anything
DELETE /users/123 | Yes | Deleting twice = same as once
PUT /users/123 {name: "Bob"} | Yes | Setting to same value repeatedly
POST /orders | NO! | Creates new order each time
POST /payments | Must make it so! | Use idempotency key

How to Make Operations Idempotent

1. Idempotency Key

Client sends: POST /payment {amount: 100, idempotency_key: "abc123"}
Server checks: Have I seen "abc123" before?
- No: Process payment, store key
- Yes: Return previous result (don't charge again!)

2. Database Constraints

```
-- Unique constraint prevents duplicates
INSERT INTO payments (id, amount, user_id)
VALUES ('abc123', 100, 456)
ON CONFLICT (id) DO NOTHING;
```

Retry Strategies

1. Simple Retry
* Try again immediately
* Problem: If server is overloaded, this makes it worse!

2. Exponential Backoff

Attempt 1: Wait 1 second
Attempt 2: Wait 2 seconds
Attempt 3: Wait 4 seconds
Attempt 4: Wait 8 seconds
(Doubles each time!)

3. Exponential Backoff with Jitter
* Add random time to avoid all clients retrying simultaneously
* Attempt 1: 1s + random(0-0.5s)
* Attempt 2: 2s + random(0-1s)

Memory Trick

IDEMPOTENT = I Do Everything More than once but the outcome remains the same regardless of my Patience of ENTering it multiple Times

Today's Quiz

1. Is "transfer $50 from A to B" idempotent? How would you make it so?
2. Why is exponential backoff better than immediate retry?
3. What is "jitter" and why do we need it?

DAY 38: Circuit Breaker Pattern

The Big Idea

A circuit breaker in software works like a circuit breaker in your home. When there's an electrical problem, it trips and cuts power to prevent fire. In software, it stops calling a failing service to prevent cascading failures!

The Problem

```
Service A calls Service B.
Service B is down.
Without circuit breaker:
  Service A keeps trying... waiting... timing out...
  Service A gets slow...
  Service C (which depends on A) gets slow too...
  EVERYTHING cascades and crashes!
```

Circuit Breaker States

```
[CLOSED] -- too many failures --> [OPEN] -- timeout expires --> [HALF-OPEN]
   |                          |                          |
   | Normal operation         | Reject all requests      | Allow few test requests
   | Requests pass through    | Return error immediately |
   |                          |                          |
   | <-- success -- [HALF-OPEN] -- if tests pass --> back to CLOSED
                              -- if tests fail --> back to OPEN
```

CLOSED (Normal):
* Everything working fine
* Requests pass through
* Count failures

OPEN (Protection mode):
* Too many failures detected!
* Reject requests immediately (don't even try)
* Wait for timeout period

HALF-OPEN (Testing):
* After timeout, try a few requests
* If they succeed -> CLOSE (back to normal!)
* If they fail -> OPEN again (still broken)

Memory Trick

Like a doctor:
* CLOSED = Patient is healthy, normal activities
* OPEN = Patient is sick, bed rest (no activities)
* HALF-OPEN = Patient trying light exercise (testing if better)

Real Example: Netflix

* Netflix has 700+ microservices
* If one dies, circuit breaker prevents cascade
* Instead of waiting, show cached/default content
* "Sorry, recommendations unavailable" is better than entire Netflix down!

Settings to Configure

* Failure threshold: How many failures before opening? (e.g., 5 failures)
* Timeout: How long to stay open? (e.g., 30 seconds)
* Success threshold: How many successes in half-open to close? (e.g., 3 successes)

Today’s Quiz

1. What happens in the HALF-OPEN state?
2. Why is "fail fast" better than "wait and timeout"?
3. How does Netflix use circuit breakers?

DAY 39: Saga Pattern - Distributed Transactions

The Big Idea

In microservices, a single business transaction might span multiple services. If one step fails, you need to UNDO all previous steps. A Saga manages this!

The Problem

> Booking a Trip:
> 1. Book Flight ?
> 2. Book Hotel ?
> 3. Book Car ? (FAILED!)
>
> Now what? Need to cancel flight AND hotel!

In a monolith, you'd use database transactions (rollback).
In microservices, services have SEPARATE databases - can't rollback!

Saga Solution

Each step has a compensating action (undo):

```
Step 1: Book Flight    | Undo: Cancel Flight
Step 2: Book Hotel     | Undo: Cancel Hotel
Step 3: Book Car       | Undo: Cancel Car

If Step 3 fails:
  - Run Undo for Step 2 (Cancel Hotel)
  - Run Undo for Step 1 (Cancel Flight)
```

Two Saga Approaches

1. Choreography (No central coordinator)

```
Services communicate through events:
Flight: "Flight Booked!" --> Hotel hears, books hotel
Hotel: "Hotel Booked!" --> Car hears, tries to book car
Car: "Car Failed!" --> Hotel hears, cancels hotel
Hotel: "Hotel Cancelled!" --> Flight hears, cancels flight
```

Like a dance where everyone knows their part!

2. Orchestration (Central coordinator)

```
[Saga Orchestrator]
  --> "Book flight" --> Flight Service: "Done!"
  --> "Book hotel" --> Hotel Service: "Done!"
  --> "Book car" --> Car Service: "Failed!"
  --> "Cancel hotel" --> Hotel Service: "Cancelled!"
  --> "Cancel flight" --> Flight Service: "Cancelled!"
```

Like a conductor leading an orchestra!

Memory Trick

* Choreography = Everyone dances on their own (no director)
* Orchestration = One conductor tells everyone what to play

Real Example: E-commerce Order

```
1. Reserve Inventory | Undo: Release Inventory
2. Process Payment   | Undo: Refund Payment
3. Ship Order        | Undo: Cancel Shipment
4. Send Confirmation | Undo: Send Cancellation Email
```

Today’s Quiz

1. When would you use Choreography vs Orchestration?
2. What is a compensating transaction?
3. Design a saga for a money transfer between two banks.

DAY 40: System Design Trade-offs Summary

The Big Idea

System design is ALL about trade-offs. There’s never a perfect solution - only the BEST solution for YOUR specific situation!

Key Trade-offs

1. Consistency vs Availability (CAP)
* Strong consistency = Slower but accurate
* High availability = Faster but might be stale

2. Latency vs Throughput
* Optimize for speed of individual request?
* Or optimize for total requests handled?

3. SQL vs NoSQL
* Structure & ACID vs Flexibility & Speed

4. Monolith vs Microservices
* Simplicity vs Scalability

5. Batch vs Real-time
* Efficiency vs Immediacy

6. Cache vs Freshness
* Speed vs Accuracy (cached data might be old)

7. Replication vs Cost
* More copies = More safety but more expensive

8. Simplicity vs Performance
* Simple code that’s maintainable vs Complex optimized code

The Golden Rule of Trade-offs

Always ask: "What are we optimizing for?"
* Social media: Optimize for availability
* Banking: Optimize for consistency
* Gaming: Optimize for latency
* Big data: Optimize for throughput

Memory Trick

TRADE = Think Really About Design Extremes
Every choice has a cost. Know what you’re paying!

Phase 2 Checklist

* [ ] Availability & Reliability
* [ ] Consistency Patterns
* [ ] Hashing & Checksums
* [ ] WebSockets & Real-Time
* [ ] Authentication & Authorization
* [ ] Logging, Monitoring & Alerting
* [ ] Storage Systems
* [ ] Streaming & Content Delivery
* [ ] Distributed Systems
* [ ] Leader Election
* [ ] Event-Driven Architecture
* [ ] Search Systems
* [ ] Data Partitioning
* [ ] Batch vs Stream Processing
* [ ] API Gateway
* [ ] Database Types
* [ ] Idempotency & Retries
* [ ] Circuit Breaker
* [ ] Saga Pattern
* [ ] Trade-offs

MINI PROJECT: Design a Food Delivery System

Design a system like UberEats/DoorDash with:
* User browses restaurants and menus
* User places order and pays
* Restaurant accepts order and prepares food
* Driver is assigned and picks up food

* Real-time tracking of delivery
* Rating system after delivery

Identify: Which databases? Which patterns? What trade-offs?

PHASE 3: BUILDING BLOCKS (Days 41-60)

Components & Design Patterns

DAY 41: Unique ID Generation

The Big Idea

Every item in a system needs a unique ID (like your social security number). But in a distributed system with billions of items, how do you make sure no two things get the same ID?

The Challenge

* Multiple servers creating IDs simultaneously
* Need to be unique across ALL servers
* Need to be fast (can’t check against all existing IDs)
* Sometimes need to be sortable by time

Solutions

1. UUID (Universally Unique Identifier)

    Example: 550e8400-e29b-41d4-a716-446655440000
    - 128 bits, extremely unlikely to collide
    - No coordination needed between servers
    - Problem: Long, not sortable, bad for database indexes

2. Auto-Increment with Database

Server 1: IDs = 1, 3, 5, 7... (odd numbers)
Server 2: IDs = 2, 4, 6, 8... (even numbers)
- Simple but limited to how many servers
- Need coordination

3. Twitter's Snowflake ID

64-bit ID structure:
| 1 bit unused | 41 bits timestamp | 10 bits machine ID | 12 bits sequence |

- Timestamp: milliseconds (gives ~69 years)
- Machine ID: which server (1024 machines)
- Sequence: counter per millisecond (4096 per ms per machine)
- SORTABLE! IDs created later have higher values
- Each machine generates independently!

4. Instagram's ID

- Similar to Snowflake
- 41 bits: timestamp in milliseconds
- 13 bits: shard ID (which database)
- 10 bits: auto-incrementing sequence

Memory Trick

SNOWFLAKE = Sorted, No collisions, Organized With Fast Local Assignment, Known Everywhere

Which to Choose?

Need | Best Choice
Simple, any size | UUID
Sortable, fast | Snowflake
Human-readable | Custom (like "USR-001")
Super compact | Auto-increment (if single DB)

Today's Quiz

1. Why can't you just use auto-increment in distributed systems?
2. What makes Snowflake IDs sortable?
3. Design an ID system for a social media app with 10,000 posts/second.

DAY 42: Bloom Filters - Probably Yes, Definitely No

The Big Idea

A Bloom Filter is a space-efficient data structure that tells you:
* "Definitely NOT in the set" (100% sure)
* "PROBABLY in the set" (might be wrong)

Like a bouncer who might accidentally let a stranger in, but NEVER keeps a VIP out!

How It Works

1. Create a bit array of size m (all zeros): [0,0,0,0,0,0,0,0,0,0]

2. To ADD "apple":
   hash1("apple") = 3, hash2("apple") = 7
   Set bits 3 and 7: [0,0,0,1,0,0,0,1,0,0]

3. To ADD "banana":
   hash1("banana") = 1, hash2("banana") = 7
   Set bits 1 and 7: [0,1,0,1,0,0,0,1,0,0]

4. To CHECK "cherry":
   hash1("cherry") = 3, hash2("cherry") = 5
   Bit 3 = 1 ?, Bit 5 = 0 ?
   Result: DEFINITELY NOT in set!

5. To CHECK "apple":
   hash1("apple") = 3, hash2("apple") = 7
   Bit 3 = 1 ?, Bit 7 = 1 ?
   Result: PROBABLY in set (might be false positive)

Real Use Cases

* Google Chrome: Is this URL malicious? (Check bloom filter before expensive lookup)
* Databases: Is this key in this file? (Avoid reading disk if definitely not)
* CDN: Has this content been cached?
* Social media: Has this user seen this post?

Memory Trick

BLOOM = Blocks Lookups On Obviously Missing data

Trade-offs

* Pro: Very memory efficient (1 bit per element vs storing actual data)
* Pro: O(1) lookup time (constant speed)
* Con: False positives possible
* Con: Can’t delete elements (or use Counting Bloom Filter)

Today’s Quiz

1. Can a Bloom Filter give false negatives?
2. Why is a Bloom Filter used before database lookups?
3. What happens to accuracy as you add more elements?

DAY 43: Geospatial Indexing - Location-Based Systems

The Big Idea

How does Uber find the nearest drivers? How does Google Maps show restaurants near you? The answer: Geospatial Indexing - special ways to organize location data!

The Problem

You’re at location (40.7, -74.0) and want to find all restaurants within 1 mile.
Checking ALL restaurants in the database = TOO SLOW!

Solutions

1. Geohashing
Converts location to a string:

> (40.748817, -73.985428) --> "dr5ru7"
> Nearby locations share the same PREFIX!
> "dr5ru7" and "dr5ru6" are neighbors!

Like a zip code but more precise:
* "d" = continent
* "dr" = region
* "dr5" = city area
* "dr5r" = neighborhood
* "dr5ru" = block
* "dr5ru7" = exact spot

2. Quadtree

Split the world into 4 quadrants recursively:

```
[World]
??? [NW quarter]
?   ??? [NW-NW] (few items, stop splitting)
?   ??? [NW-SE] (many items, keep splitting...)
??? [NE quarter]
??? [SW quarter]
??? [SE quarter]
```

Dense areas get split more = more precision where needed!

3. R-Tree
* Groups nearby objects in bounding boxes
* Used in PostGIS (PostgreSQL extension)
* Great for: "Find all restaurants in this rectangle"

Memory Trick

GEO = Grids Enable Ordering (of locations)

Real Examples

* Uber: Geohash to find nearby drivers
* Tinder: Geohash to find nearby users
* DoorDash: Quadtree to find nearby restaurants
* Google Maps: R-tree for map rendering

Today’s Quiz

1. Why is geohashing fast for "nearby" queries?
2. When would you choose Quadtree over Geohash?
3. Design a system to find the 5 nearest gas stations.

DAY 44: Counters at Scale

The Big Idea

Counting sounds simple, right? But what if 100,000 people "like" a post at the SAME SECOND? A simple counter would break! Let’s learn how to count at massive scale.

The Problem

Post has 999,999 likes.
1000 people click "like" simultaneously.
Without careful design: All read 999,999 and write 1,000,000
Result: 1,000,000 instead of 1,000,999!

Solutions

1. Atomic Operations

Instead of: READ count, ADD 1, WRITE new count
Do: INCREMENT count BY 1 (single atomic operation)
Database handles the locking!
Redis: INCR likes:post123

2. Sharded Counters
* Split one counter into many sub-counters
* Each sub-counter handles a portion of increments

Total Likes = counter_1 + counter_2 + counter_3 + counter_4
Counter 1: 250,000
Counter 2: 250,333
Counter 3: 250,100
Counter 4: 250,566
Total: ~1,000,999

* Reduces contention (less fighting over one number)!

3. Approximate Counting (HyperLogLog)
* When exact count doesn’t matter
* Uses very little memory
* Example: "This post has ~1M views" (not exactly 1,000,247)
* Redis: PFADD and PFCOUNT

Memory Trick

Think of counting like voting:
* Atomic = One ballot box, everyone waits in line
* Sharded = Multiple ballot boxes, faster!
* Approximate = Exit polls (close enough)

Real Examples

* YouTube view count: Eventually consistent (shows "1.2M views" not exact)
* Twitter like count: Sharded counters
* Reddit upvotes: Cached and periodically synced